EECS 348 Group 10

Arithmetic Expression Evaluator
Software Development Plan
Version <1.0>

[Note: The following template is provided for use with the Unified Process for EDUcation. Text enclosed in square brackets and displayed in blue italics (style=InfoBlue) is included to provide guidance to the author and should be deleted before publishing the document. A paragraph entered following this style will automatically be set to normal (style=Body Text).]

[To customize automatic fields in Microsoft Word (which display a gray background when selected), select File>Properties and replace the Title, Subject and Company fields with the appropriate information for this document. After closing the dialog, automatic fields may be updated throughout the document by selecting Edit>Select All (or Ctrl-A) and pressing F9, or simply click on the field and press F9. This must be done separately for Headers and Footers. Alt-F9 will toggle between displaying the field names and the field contents. See Word help for more information on working with fields.] Marked (shaded) areas: items that are OK to leave out.

Revision History

Date	Version	Description	Author
02/16/2026	1.0	Initial commits / group discussion	Tate Meyer
02/18/2026	1.1	Project overview is complete	Jaycob Campos
02/21/2026	1.2	Introduction (Section 1) complete; 1.4 requires revision.	Ayushma Koirala
02/21/2026	1.3	Final Overview and completing section 4	Maddi Clark

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 5

1.5 Overview 5

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 6

2.3 Project Deliverables 6

2.4 Evolution of the Software Development Plan 6

3. Project Organization 6

3.1 Organizational Structure 6

3.2 External Interfaces 7

3.3 Roles and Responsibilities 7

4. Management Process 8

4.1 Project Estimates 8

4.2 Project Plan 8

4.3 Project Monitoring and Control 9

4.4 Requirements Management 10

4.5 Quality Control 10

4.6 Reporting and Measurement 10

4.7 Risk Management 11

4.8 Configuration Management 11

5. Annexes 11

Software Development Plan

1. Introduction

This document presents the Software Development Plan for the Arithmetic Expression Evaluator project. It outlines the overall approach, structure, and management strategy that will guide the successful development and delivery of the project.

1.1 Purpose

The purpose of this Software Development Plan is to define the development approach, organization, schedule, and management strategy for the Arithmetic Expression Evaluator project. This document serves as a guide for the project team to plan, execute, monitor, and control the development process.

- This plan will be used by the project manager to track progress and allocate responsibilities
- Team members will use this document to understand their tasks, deadlines, and dependencies. It ensures that the final product meets the specified requirements and quality standards.

1.2 Scope

This Software Development Plan applies to the development of the Arithmetic Expression Evaluator in C++. The system will parse and evaluate arithmetic expressions containing numeric constants and the following operators:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulo (%)
- Exponentiation (**)
- Unary + and -

The program will correctly implement operator precedence, associativity rules, and parenthesis handling. It will include proper error handling for invalid expressions and runtime errors such as division by zero.

This document covers the full development lifecycle of the project including planning, requirements analysis, design, implementation, testing, documentation, and delivery

1.3 Definitions, Acronyms, and Abbreviations

- **Parser** – A component that analyzes an input expression and determines its structure according to defined rules.

- **Tokenizer** – A module that breaks the input string into meaningful symbols (tokens) such as numbers and operators.
- **Operator Precedence** – The rules that determine the order in which operations are performed.
- **Associativity** – The rule that determines the order of evaluation when operators have the same precedence (left-to-right or right-to-left).
- **PEMDAS** – Parentheses, Exponents, Multiplication, Division, Addition, Subtraction.
- **CLI** – Command Line Interface

1.4 References

*[This subsection provides a complete list of all documents referenced elsewhere in the **Software Development Plan**. Identify each document by title, report number if applicable, date, and publishing organization. Specify the sources from which the references can be obtained. This information may be provided by reference to an appendix or to another document.*

*For the **Software Development Plan**, the list of referenced artifacts includes:*

- Iteration Plans
- *Development Case: EECS 348 Term Project Description – Arithmetic Expression Evaluator in C++*
- *Vision [you may prepare a vision statement of your own: what your vision for the project is]*
- *Glossary*
- Software Development Plan Template

1.5 Overview

This Software Development Plan is organized into several sections.

- Project Overview (Section 2) -Provides a detailed overview of the project objectives and deliverables.
- Project Organization (Section 3) -Describes the project organization, roles, and responsibilities of team members.
- Management Process (Section 4) -Outlines the management process, including scheduling, risk management, quality control, and configuration management.
- Annex (Section 5) - Additional supporting materials are provided in the annex section.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

The purpose of this project is to design and develop an arithmetic expression evaluator as a component of a larger compiler product for language L. The parser will be built in C++ and will be able to parse and compute arithmetic expressions following PEMDAS. The scope of the project covers design, implementation, testing, and documentation for the arithmetic parser. It includes the operators +, -, *, /, %, and **, unary operators, parenthetical grouping, numbers, and error handling. The objective is to develop a C++ program that correctly evaluates all valid arithmetic expressions and returns clear error messages for invalid inputs. All required project artifacts should align with one another throughout the development process. Deliverables include a project management plan, requirements documents, design document, test plan, a well-documented

C++ program, and a user manual with usage examples.

[A brief description of the purpose and objectives of this project and a brief description of what deliverables the project is expected to deliver.]

2.2 Assumptions and Constraints

Assumption are that all members have a working knowledge of C++ and object-oriented programming basics. The inputs for the parser will only contain operators and operands specified in the project requirements. Team members will have constant access to the GitHub repository and development environments that support C++. Constraints are the time we have to complete this arithmetic parser. The equipment is restricted to personal machines or KU computers. The arithmetic parser has six operators and doesn't need to support variables or functions. The project has to be coded in C++.

[A list of assumptions that this plan is based and any constraints, for example. staff, equipment, schedule, that apply to the project.]

2.3 Project Deliverables

Project artifacts that will be created during the project are project management plan, requirements document, design document, test plan, C++ source code, user manual/ README. All deliverables will be submitted by the GitHub repository by the end of the specified project phase. Deliverables for each project phase are identified in the Development Case. Deliverables are delivered towards the end of the iteration, as specified in section *4.2.4 Project Schedule*.

[A list of the artifacts to be created during the project, including target delivery dates. The text below is provided as an example.] Requirements, design specs, test cases, code

2.4 Evolution of the Software Development Plan

Version	Description	Date
1.0	Initial Commits	Feb. 16
1.1	Project Overview Edits	Feb. 18
1.2	Introduction Complete	Feb. 21
1.3		
1.4		

*[A table of proposed versions of the **Software Development Plan**, and the criteria for the unscheduled revision and reissue of this plan. The text below is provided as an example.]*

The *Software Development Plan* will be revised prior to the start of each Iteration phase.

3. Project Organization

3.1 Organizational Structure

[Describe the organizational structure of the project team, including management and other review authorities.]

The project team consists of six members working collaboratively under the UPEDU framework. While each member has an assigned primary role, the overall structure is cooperative, allowing flexibility and shared responsibility throughout development.

- The Project Manager oversees coordination, scheduling, and progress tracking.
- The Software Architect provides guidance on system design decisions.
- Test Engineers focus on verification and quality assurance.
- Documentation Managers maintain and organize project documentation.

The course instructor and teaching assistants serve as external review authorities, providing feedback and evaluating project deliverables.

3.2 External Interfaces

[Describe how the project interfaces with external groups. For each external group, identify the internal and external contact names. This should include responsibilities related to deployment and acceptance of the product.]

The primary external stakeholders for this project are the course instructor and the teaching assistants. They provide guidance, clarify requirements, and evaluate deliverables throughout the semester. Most communication with them will take place during class/lab sessions or through email if additional clarification is needed.

GitHub will serve as the team's primary collaboration and version control platform. It will function as the central repository for source code, documentation, issue tracking, and submission of deliverables. Using GitHub consistently ensures that all team members are working with the most up-to-date project materials. Additional team communication will occur through Microsoft Teams and scheduled group meetings.

As this is an academic project, final acceptance of the product will be determined by the instructor based on the grading rubric and the stated project requirements.

3.3 Roles and Responsibilities *[the more details here, the easier your job; include contact info, availability info, expertise, …]*

[Identify the project organizational units that will be responsible for each of the disciplines, workflow details, and supporting processes. The text below is provided as an example.]

Person	**Unified Process for Education Role**
Tate Meyer	Project Manager / Programmer
Avery Richardson	Test Engineer / Programmer
Nikki Sotoodi	Test Engineer
Maddi Clark	Documentation Manager
Ayushma Koirala	Documentation Manger

Jaycob Campos	Software Architect/Designer

Anyone on the project can perform Any Role activities.

4. Management Process

4.1 Project Estimates

[Provide the estimated cost and schedule for the project, as well as the basis for those estimates, and the points and circumstances in the project when re-estimation will occur.]

Seeing as we are students doing this project for credit, labor cost will be 0. We will also be using all free tools to create our project. We are provided with a schedule with due dates. These dates include:

- Project Plan due Feb. 22
- Software Requirements and initial user manual due Mar. 15
- Design and Architecture due Apr. 5
- Final Implementation, Final User Manual, and Final test cases due May 7

4.2 Project Plan

[This section contains the schedule and resources for the project.]Project artifact as well as iteration schedules]

4.2.1 Phase Plan

[Include the following:

- *a Gantt chart showing the allocation of time to the project phases (Not necessarily detailed to the activity level; this type of Gantt Chart is providing along with the Iteration Plans themselves; Provide an Overview of the project Timeline with the major miles stones]*
- *identify **major milestones** with their achievement criteria*

Define any important release points and demos.]

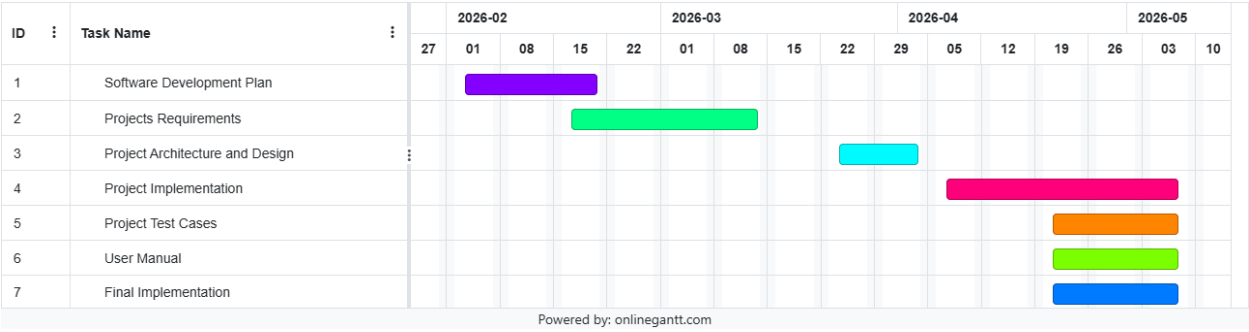
*[If available, refer to the related **Iteration Plan Documents** for more details]*

Milestones:

- Completed Software Development Plan
- Completed Requirements Document
- Completed Design/Architecture
    - Various UML diagrams showing the architecture and how different aspects of the software work together
- Completed Implementation
    - Can correctly evaluate strings with +. -. *, /, %, and **
    - Can correctly evaluate parentheses
    - Can correctly evaluate positive and negative numbers
    - Any additional features
- Completed User Manual

- Depicts how to correctly use the software, preferably with photo examples

- Completed all test cases showing that the software works as intended



4.2.2 Iteration Objectives

*[Briefly list the objectives to be accomplished for each of the iterations and Refer to the related **Iteration Plan Documents** for more details.]*

4.2.3 Releases

[A brief description of each software release and whether it's demo, beta, and so on.]

4.2.4 Project Schedule

[Diagrams or tables showing target dates for completion of iterations and phases, release points, demos, and other milestones.] [Limit to major project milestone, e.g., requirements, design, implementaiotn, and testing] March 15 – Requirements complete and initial iteration of the user manual

April 5 – Design and Architecture done

May 7 – Final implementation, final user manual, final test cases completed

4.2.5 Project Resourcing

[Identify the numbers and type of staff required here, including any special skills or experience, scheduled by project phase or iteration.

List any special training project team members will require, with target dates for when this training should be completed.]

5 total members consisting of Project Manager, Programmers, Test Engineers, Documentation Managers, Software Architects, and Design. Skills required will be C++ experience, Git and GitHub, and design experience is preferred. No extra training should be required.

4.3 Project Monitoring and Control

[The following is a checklist of items to consider:

- *Requirements Management: Specify the information and control mechanisms which will be collected and used for measuring, reporting, and controlling changes to the product requirements.*
- *Quality Control: Describe the timing and methods to be used to control the quality of the project deliverables and how to take corrective action when required. Include techniques, metrics, criteria, and procedures used for evaluation— this will include walkthroughs, inspections, and reviews. Note that this is in addition to the Test Plan, which is not enclosed in the Software Development Plan.*

- *Reporting and Measurement: Describe reports to be generated. Specify which metrics should be collected and why.* ***OR*** *if available, refer to the* ***Project Measurements and Project Measurements*** *document* Reports will be generated after each main feature is implemented. Metrics will include whether or not the sample expressions are evaluated correctly and if errors are handled correctly.
- *Risk Management: Describe the approach that will be used to identify, analyze, prioritize, monitor and mitigate risks. If available, refer to the* ***Risk List*** *document.*
- *Configuration Management: Describe the process by which problems and changes are submitted, reviewed, and dispositioned. Describe how project or product artifacts are to be named, marked, and numbered, including system software, plans, models, components, test software, results and data, executables, and so on. Describe retention policies, and the back-up, disaster, and recovery plans.* ***OR*** *if Available, Refer to the* ***Configuration Management Plan*** *document*

The text that follows is provided as an example.]

4.4 Requirements Management

The requirements for this system are captured in the Vision document. Requested changes to requirements are captured in Change Requests, and are approved as part of the Configuration Management process.

4.5 Quality Control

Defects will be recorded and tracked as Change Requests, and defect metrics will be gathered (see Reporting and Measurement below).

All deliverables are required to go through the appropriate review process, as described in the Development Case. The review is required to ensure that each deliverable is of acceptable quality, using guidelines and checklists.

Any defects found during review which are not corrected prior to releasing for integration must be captured as Change Requests so that they are not forgotten.

4.6 Reporting and Measurement

Updated schedule estimates, and metrics summary reports, will be generated at the end of each iteration.

The Minimal Set of Metrics, as described in the RUP Guidelines: Metrics will be gathered on a weekly basis. These include:

Earned value for completed tasks. This is used to re-estimate the schedule and budget for the remainder of the project, and/or to identify need for scope changes.

Total defects open and closed – shown as a trend graph. This is used to help estimate the effort remaining to correct defects.

Acceptance test cases passing – shown as a trend graph. This is used to demonstrate progress to stakeholders.

Refer to the Project Measurements Document (AAA-BBB-X.Y.doc) for detailed information.

4.7 Risk Management

Risks will be identified in Inception Phase using the steps identified in the RUP for Small Projects activity "Identify and Assess Risks". Project risk is evaluated at least once per iteration and documented in this table.

Refer to the Risk List Document (CCC-DDD-X.Y.doc) for detailed information.

4.8 Configuration Management

Appropriate tools will be selected which provide a database of Change Requests and a controlled versioned repository of project artifacts.

All source code, test scripts, and data files are included in baselines. Documentation related to the source code is also included in the baseline, such as design documentation. All customer deliverable artifacts are included in the final baseline of the iteration, including executables.

The Change Requests are reviewed and approved by one member of the project, the Change Control Manager role.

Refer to the Configuration Management Plan (EEE-FFF-X.Y.doc) for detailed information.

5. Annexes

*[Additional material of use to the reader of the **Software Development Plan**. Reference or include any project technical standards and plans which apply to this project. This typically includes the Programming Guidelines, Design Guidelines, and other process guidelines. The text that follows is provided as an example.]*

The project will follow the UPEDU process.

Other applicable process plans are listed in the references section, including Programming Guidelines.